



National University of Computer & Emerging Sciences, Karachi

Fall -2025 CS/CY/SE-Department

Assignment- 2

18th September 2025



Course Code: CS2009	Course Name: Design & Analysis of Algorithms
Instructor Name:	
Date of Submission	30 th September 2025

Instructions:

Max Marks: 55

- Make your Assignments on Full Scale Papers, Typed Assignments will not be acceptable:
- 20% penalty for 1 day late
- 40% penalty for 2 days late
- Submission not allowed afterwards

1) A large e-commerce company needs to process a massive list of orders based on their total value. The orders are represented as integers, and the company wants to prioritize the orders with the highest value. Design an efficient algorithm to sort the orders in descending order of their total value using Quick Sort (Divide and Conquer) and compute the time complexity of your designed algorithm. Analyse and state the **time complexity** of your algorithm (best, average, and worst case).

(10 Marks)

Solution

Assignment 2
Question 1
Quick Sort

Pseudocode

Quicksort($A, low, high$)

if $low < high$

$pi = \text{Partition}(A, low, high)$

 Quicksort($A, low, pi - 1$)

 Quicksort($A, pi + 1, high$)

Partition($A, low, high$)

$pivot = A[high]$

$i = low - 1$

 for j from low to $high - 1$

 if $A[j] \geq pivot$

$i = i + 1$

 swap $A[i]$ and $A[j]$

 swap $A[i + 1]$ and $A[high]$

 return $i + 1$

$A = [120, 300, 150, 500, 250]$, low = 0, high = 4

pivot = $A[4] = 250$, $i = -1$

$J = 0$, $120 < 250$, $i = 1$

$J = 1$: $300 > 250$, $j = 0$, swap $A[0]$ & $A[1] \rightarrow [300, 120, 150, 500, 250]$

$J = 2$: $150 < 250$, $i = 0$

$J = 3$: $500 > 250$, $j = 1$, swap $A[1]$ & $A[3] \rightarrow [300, 500, 150, 120, 250]$

loop ends, swap $A[i+1] = A[2]$ at pivot $A[4]$

$\hookrightarrow [300, 500, 250, 120, 150]$

Pivot final index $p_i = 2$

$\rightarrow$ recursively sort left in

left $A[0..1] = [300, 500]$

right $A[3..4] = [120, 150]$

$\rightarrow$ Sort left $[300, 500]$

Pivot = 500, partition results in $[500, 300]$

$\rightarrow$ Sort right $[120, 150]$

Pivot = 150, partition results in $[150, 120]$

Final array $[500, 300, 250, 150, 120]$

Time Complexity

$T(n)$ on 'n' elements (only comparisons + swaps $\approx \Theta(n)$ per partition)

Best Case :- Pivot always splits arrays exactly in half.

$$T(n) = 2T\left(\frac{n}{2}\right) + \Theta(n)$$

by Master Theorem

$$a=2, b=2, d=1$$

$$a < b^d$$

Case 2

$$2 < 2^1$$

$$\Theta(n^d \lg n) \rightarrow \Theta(n \lg n)$$

Worst Case :- Pivot is always the smallest or largest element (unbalanced)

Two array size of 0 and $n-1$. $T(n) = T(n-1) + \Theta(n)$
 $\Theta(n^2)$

2) A university's registrar office needs to process a list of student scores for a particular exam. The list of scores is as follows: [59,6, 35, 12, 27, 9, 1, 18, 5, 31,16]. To facilitate grading and feedback, the office wants to sort the list of scores in ascending order. **(10 Marks)**

1. By applying Merge Sort generate a step-by-step algorithm to sort the list of student scores

```
def merge_sort(arr):
    if len(arr) > 1:
        # Step 1: Divide the array into two halves
        mid = len(arr) // 2
        left_half = arr[:mid]
        right_half = arr[mid:]

        # Step 2: Recursively sort both halves
        merge_sort(left_half)
        merge_sort(right_half)

        # Step 3: Merge the sorted halves
        i = j = k = 0
        # Compare elements and merge
        while i < len(left_half) and j < len(right_half):
            if left_half[i] < right_half[j]:
                arr[k] = left_half[i]
                i += 1
            else:
                arr[k] = right_half[j]
                j += 1
            k += 1
        # Copy remaining elements of left_half (if any)
        while i < len(left_half):
            arr[k] = left_half[i]
            i += 1
            k += 1
        # Copy remaining elements of right_half (if any)
        while j < len(right_half):
            arr[k] = right_half[j]
            j += 1
            k += 1
```

2. Show the detailed steps of the Merge Sort algorithm to sort the list in ascending order.

Sun Mon Tue Wed Thu Fri Sat

Merge Sort

Original array 59, 6, 35, 12, 27, 9, 1, 18, 5, 31, 16

Split whole array
split at mid = 5

Left [0-5] $\rightarrow$ [59, 6, 35, 12, 27, 9]
Right [6-10] $\rightarrow$ [1, 18, 5, 31, 16]

$\rightarrow$ Left [0-5] [59, 6, 35, 12, 27, 9]
split at mid = 2

left-left [0-2] $\rightarrow$ [59, 6, 35]
left-right [3-5] $\rightarrow$ [12, 27, 9]

$\rightarrow$ Left-left [0-2] :- [59, 6, 35]
split at mid = 1

[0-0] [59]
[1-2] [6, 35]

[59] is base single element
[6, 35] splits to 6 and 35 (base now) and merge

- Merge 6 and 35, compare $6 \leq 35 \rightarrow [6, 35]$
- Merge [59] and [6, 35]
 - Compare 59 and 6, 6 is smaller $\rightarrow$ pick 6
 - Now compare 59 and 35 $\rightarrow$ 35 is smaller $\rightarrow$ pick 35
 - Left over 59 $\rightarrow$ append $\rightarrow$ 6, 35, 59

$\rightarrow$ Left-right [3-5] :- [12, 27, 9]
split at mid = 4

Left Right-left [3-4] $\rightarrow$ [12, 27] split to 12 and 27
and merge 12, 27

[5-5] $\rightarrow$ 9 (base)

Now merge [12, 27] and 9

Compare 12 and 9 $\rightarrow$ 9 is smaller pick 9
9, 12, 27

Merge left-left and right-right

Merge [6, 35, 59] and [9, 12, 27]

Compare 6 and 9, pick 6

Compare 35 and 9, pick 9

Compare 35 and 12, pick 12

Compare 35 and 27 $\rightarrow$ pick 27

left 35, 59

Append

6, 9, 12, 27, 35, 59

Whole left half is sorted

Right subtree [6-10] [1, 18, 5, 31, 16]

split at mid = 8

Right-left [6-8] → [1, 18, 5]

Right-right [9-10] → [31, 16]

→ Right-left [6-8] → [1, 18, 5]

split at mid = 7

RL-left [6-7] → [1, 18] split to 1 and 18 and merge.

RL-right 8-8 → [5]

Merge [1, 18] and [5]

- Compare 1 and 5, pick 1

18 and 5, n = 5

[1, 5, 18]

→ Right-right → [9, 10] : [31, 16]

split to [31] and [16] and merge

Compare 31 and 16 and merge.

Merge Right-left and Right-right

[1, 5, 18] and [16, 31]

- Compare 1 and 16, pick 1

5 ~ 16, pick 5

18 ~ 16, pick 16

18 ~ 31 → pick 18.

[1, 5, 16, 18, 31]

Final merge :- (merge left and right half)

[6, 9, 12, 27, 35, 59] and [1, 5, 16, 18, 31]

Compare and merge as above.

Final list → Sorted = [1, 5, 6, 9, 12, 16, 18, 27, 31, 35, 39]

3. Consider a variation of the Merge Sort algorithm that divides the input into four parts instead of two. Analyse the overall asymptotic running time of this modified algorithm. Write the recurrence relation for this modified algorithm.

Solution

Instead of splitting the input into two halves, the algorithm:

1. Divides the input array of size n into **four** equal parts (assume for simplicity n is a power of 4), each of size $n/4$
2. Recursively sorts each of the four parts.
3. Merges the four sorted parts into one sorted array.

Assume merging four sorted subarrays of total length n can be done in $\Theta(n)$ time

Base case: $T(1) = \Theta(1)$

Recurrence Relation

$$T(n) = 4T(n/4) + \Theta(n)$$

If the algorithm splits the input A size n into 4 equal parts, recursively sorts each part, then merges the four sorted parts back into one, the recurrence relation is

$$T(n) = \begin{cases} O(1) & n \leq 1, \\ 4T(n/4) + O(n), & n > 1 \end{cases} \quad \therefore \text{Merge cost } O(n)$$

by pairwise 2 way merges, merge A and B cost $n/2$, merge C and D cost $n/2$, $O(n \log 4) = O(n)$

Apply Master theorem with $a=4$, $b=4$, $d=1$

$$b^d \Rightarrow 4^1 = 4$$

$$a = b^d$$

$$4 = 4$$

$$\text{Case 2: } \Theta(n^d \log n)$$

$$= \Theta(n^1 \log n)$$

$$T(n) = n \log n$$

3) A leading bank is developing a **financial risk analysis engine** to evaluate multiple investment portfolios under various market conditions. Each portfolio is represented by a **4×4 weight matrix (W)** indicating the distribution of funds across sectors such as technology, energy, healthcare, and finance, while the expected sector returns under different market conditions are stored in another **4×4 returns matrix (R)**. To compute the overall risk-return scores, the system must efficiently perform **matrix multiplication of $W \times R$** . **(10 Marks)**

As part of this task, you are required to

- a. implement a **standard matrix multiplication algorithm** using the naïve triple-loop method and analyse its time complexity,

Algorithm Steps

1. **Input:**

Two matrices W and R, each of size 4×4

2. **Initialize:**

Create an empty result matrix C (also 4×4) and set all its elements to 0.

3. **Multiply and Add:**

For each element $C[i][j]$ in the result matrix:

- Multiply the elements in the **i-th row** of W with the corresponding elements in the **j-th column** of R
- Add all those products together.

Mathematically:

$$C[i][j] = W[i][1] \times R[1][j] + W[i][2] \times R[2][j] + W[i][3] \times R[3][j] + W[i][4] \times R[4][j]$$

Repeat this process for all rows $i=1$ to 4 and all columns $j=1$ to 4.

Output:

The final matrix CCC, which represents $W \times R$

Explanation of the Algorithm

The algorithm uses **three nested loops**:

- Outer loop → iterates over each row of **W** (i)
- Middle loop → iterates over each column of **R** (j)
- Inner loop → computes the dot product of the ith row of W and jth column of R (k)

Mathematically:

$$result[i][j] = \sum_{k=0}^3 W[i][k] * R[k][j]$$

Time Complexity Analysis

Let the matrices be of size $n \times n$

In the triple-loop approach:

Loop	Iterations	Work inside
Outer loop (i)	n	-
Middle loop (j)	n	-
Inner loop (k)	n	1 multiplication + 1 addition

So, total operations:

$$T(n) = n \times n \times n = n^3$$

Time Complexity: $T(n) = \Theta(n^3)$

- b. implement the **Strassen's matrix multiplication algorithm** using the divide-and-conquer approach by splitting the 4×4 matrices into 2×2 submatrices and applying the seven Strassen formulas (M1–M7)

Strassen's algorithm (divide-and-conquer, splitting 4×4 into 2×2 blocks)

Let the two 4×4 matrices be W and R. Split each into four 2×2 blocks:

$$W = \begin{pmatrix} A & B \\ C & D \end{pmatrix} \quad R = \begin{pmatrix} E & F \\ G & H \end{pmatrix}$$

where A,B,C,D,E,F,G,H are 2×2 submatrices.

Strassen computes seven products (each a multiplication of two 2×2 matrices) from linear combinations of the blocks:

$$M1 = (A+D)(E+H)$$

$$M2 = (C+D)E$$

$$M3 = A(F-H)$$

$$M4 = D(G-E)$$

$$M5 = (A+B)H$$

$$M6 = (C-A)(E+F)$$

$$M7 = (B-D)(G+H)$$

Then assemble the four 2×2 blocks of the product $C = W \times R$:

$$C_{11} = M1 + M4 - M5 + M7$$

$$C_{12} = M3 + M5$$

$$C_{21}=M2+M4$$

$$C_{22}=M1-M2+M3+M6$$

Finally combine C_{11} C_{12} C_{21} C_{22} into the final 4×4 result.

c. Write the recurrence relation for **Strassen's matrix multiplication algorithm**

Each level of recursion produces **7** recursive multiplications on size $n/2$, plus $\Theta(n^2)$ work for additions/subtractions of submatrices (the combination step). So the recurrence is:

$$T(n)=7T(n/2)+ \Theta(n^2)$$

d. Derive the time complexity by applying Master Theorem

$$a=7, b=2, d=2$$

$$b^d=4$$

$$a > b^d$$

$$7 > 4$$

Case 3 of Master theorem

$$\Theta(n^{\log_b a}) \rightarrow \Theta(n^{\log_2 7}) \approx \Theta(n^{2.8074})$$

e. compare the **time complexities** of both methods

So Strassen runs in $\Theta(n^{2.8074})$ which is asymptotically faster than $\Theta(n^3)$

4) Discuss all cases of Master Theorem.

(5 Marks)

Solution

The Master Theorem applies to recurrences of the form:

$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$

where:

- $a \geq 1$ = number of subproblems per recursive call
- $b > 1$ = input size reduction factor (each subproblem is size n/b)
- $f(n)$ = cost of dividing and combining (non-recursive work per call)

We compare $f(n)$ with $n^{\log_b a}$ which represents the "work from recursion."

If $f(n) \in \Theta(n^d)$ where $d \geq 0$, then

Case 1: if $a < b^d$, $T(n) = \Theta(n^d)$

Non-recursive work $f(n)$ grows **slower** than n^d .

Result: Recursion dominates.

Case 2: if $a = b^d$, $T(n) = \Theta(n^d \log n)$

Non-recursive work grows **at the same rate** (up to polylog) as recursion.

Both contribute equally.

Case 3: if $a > b^d$, $T(n) = \Theta(n^{\log_b a})$

Non-recursive work grows **faster** than recursion.

5) Using the Master Theorem, can we solve the modified recurrence relation of question.
(5 Marks)

Yes, we can.

Sun Mon Tue Wed Thu Fri Sat

If the algorithm splits the input A size n into 4 equal parts, recursively sorts each part, then merges the four sorted parts back into one, the recurrence relation is

$$T(n) = \begin{cases} O(1) & n \leq 1 \\ 4T(n/4) + O(n) & n > 1 \end{cases} \quad \therefore \text{Merge cost } O(n)$$

by pairwise 2 way merges, merge A and B cost $n/2$, merge C and D cost $n/2$, $O(n \log 4) = O(n)$

Apply Master theorem with $a=4$, $b=4$, $d=1$

$$b^d \rightarrow 4^1 = 4$$

$$a = b^d$$

$$4 = 4$$

Case 2: $\Theta(n^d \log n)$
 $= \Theta(n^1 \log n)$
 $T(n) = n \log n$

6) Use Iteration Method and Recurrence tree, to calculate the time complexity of the following (7.5 Marks)

I. $T(n) = 2T(n/2) + n^2$

Solution

Iterative / Substitution Method

$$T(n) = 2T\left(\frac{n}{2}\right) + n^2$$

$$T(n) = 2(T(n/2)) + n^2 \quad \text{--- (i)}$$

$$T\left(\frac{n}{2}\right) = 2\left(\frac{n}{4}\right) + \frac{n^2}{2} \quad \text{put in eq (i)}$$

$$= 2\left(2T\left(\frac{n}{4}\right) + \frac{n^2}{2}\right) + n^2$$

$$= 2^2 T\left(\frac{n}{4}\right) + \frac{n^2}{2} + n^2$$

$$\Rightarrow 2^2 T\left(\frac{n}{4}\right) + n^2 \left(\frac{1}{2} + 1\right)$$

$$\Rightarrow 2^3 T\left(\frac{n}{8}\right) + n^2$$

After k iteration the pattern would be

$$T(n) = 2^k T\left(\frac{n}{2^k}\right) + n^2 \sum_{i=0}^{k-1} 2^i \left(\frac{1}{2^i}\right)^2$$

$$2^i \left(\frac{1}{2^i}\right)^2 \Rightarrow 2^i \cdot \frac{1}{4^i} = \left(\frac{1}{2}\right)^i$$

$$= 2^k T\left(\frac{n}{2^k}\right) + n^2 \sum_{i=0}^{k-1} \left(\frac{1}{2}\right)^i$$

$\frac{n}{2^k} = 1, 2^k = n, k = \log_2 n$ put in equation.

$2^k \cdot T(1) \Rightarrow 2^{\log_2 n} = n$

$\sum_{i=0}^{\log_2 n} \left(\frac{1}{2}\right)^i = \frac{1 - (1/2)^{\log_2 n}}{1 - \frac{1}{2}} = 2\left(1 - \frac{1}{n}\right)$

$T(n) = \Theta(n) + n^2 \cdot 2\left(1 - \frac{1}{n}\right)$

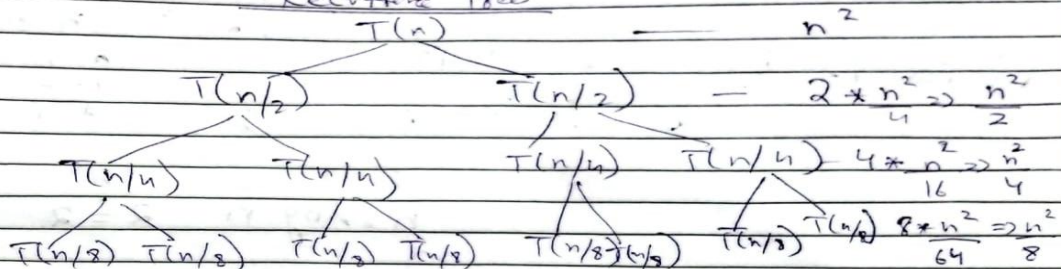
$\Rightarrow 2n^2 - 2n + \Theta(n)$

$\approx \Theta(n^2)$

We get $T(n) = \Theta(n^2)$

we get $T(n) = \Theta(n^2)$

Recursive Tree



Total = $n^2 \left(1 + \frac{1}{2} + \frac{1}{4} + \frac{1}{8} + \dots + \frac{1}{n}\right)$

Geometric series: $1 + \frac{1}{2} + \frac{1}{4} + \dots = 2$

$T(n) = n^2 \cdot 2 + O(n) = \Theta(n^2)$

II. $T(n) = 3T(n/3) + n \log^2 n$
Solution

$$T(n) = 3T\left(\frac{n}{3}\right) + n \log n \quad \text{--- eq (i)}$$

$$T\left(\frac{n}{3}\right) = 3T\left(\frac{n}{9}\right) + \frac{n}{3} \log \frac{n}{3} \Rightarrow 3T\left(\frac{n}{3^2}\right) + \frac{n}{3} \log \frac{n}{3}$$

substitute value of $T\left(\frac{n}{3}\right)$ in eq (i).

$$\Rightarrow 3T\left[3T\left(\frac{n}{3^2}\right) + \frac{n}{3} \log \frac{n}{3}\right] + n \log n$$

$$= 3^2 T \cdot \frac{n}{3^2} + n \left[\log \frac{n}{3} \right] + \log n \quad \text{--- (ii)}$$

$$T\left(\frac{n}{3^2}\right) \text{ or } T\left(\frac{n}{9}\right) \Rightarrow 3T\left(\frac{n}{3 \cdot 3} + \frac{n}{9} \log \frac{n}{9}\right)$$

Put in eq (ii)

$$\Rightarrow 3^2 \left(3T\left(\frac{n}{27}\right) + \frac{n}{9} \log \frac{n}{9} \right) + n \left[\log \frac{n}{9} \right] + \log n$$

$$\Rightarrow 3^3 T\left(\frac{n}{3^3}\right) + \frac{n}{3^2} \log_2 \left(\frac{n}{3^2}\right)$$

After k expansion

$$T(n) = 3^k T\left(\frac{n}{3^k}\right) + \sum_{i=0}^{k-1} 3^i \cdot \left(\frac{n}{3^i} \log\left(\frac{n}{3^i}\right)\right)$$

$$3^i \cdot \frac{n}{3^i} = n$$

$$n \cdot \log\left(\frac{n}{3^i}\right) = n(\log n - \log(3^i))$$

$$= n(\log n - i \log 3)$$

$$= n \sum_{i=0}^{k-1} (\log n - i \log 3) = n(k \log n - \log 3 \sum_{i=0}^{k-1} i)$$

$$= n(k \log n - \log 3 \cdot \frac{k(k-1)}{2})$$

$$\frac{n}{3^k} = 1, 3^k = n, k = \log_3 n$$

$$\log_3 n = \frac{\log_2 n}{\log_2 3}$$

$$k \log n = (\log_3 n) \log n = \frac{\log n}{\log 3} \cdot \log n = \frac{(\log n)^2}{\log 3}$$

$$\frac{\log 3}{2} k(k-1) \sim \frac{\log 3}{2} \cdot k^2 = \frac{\log 3}{2} \left(\frac{\log n}{\log 3}\right)^2 = \frac{(\log n)^2}{2 \log 3}$$

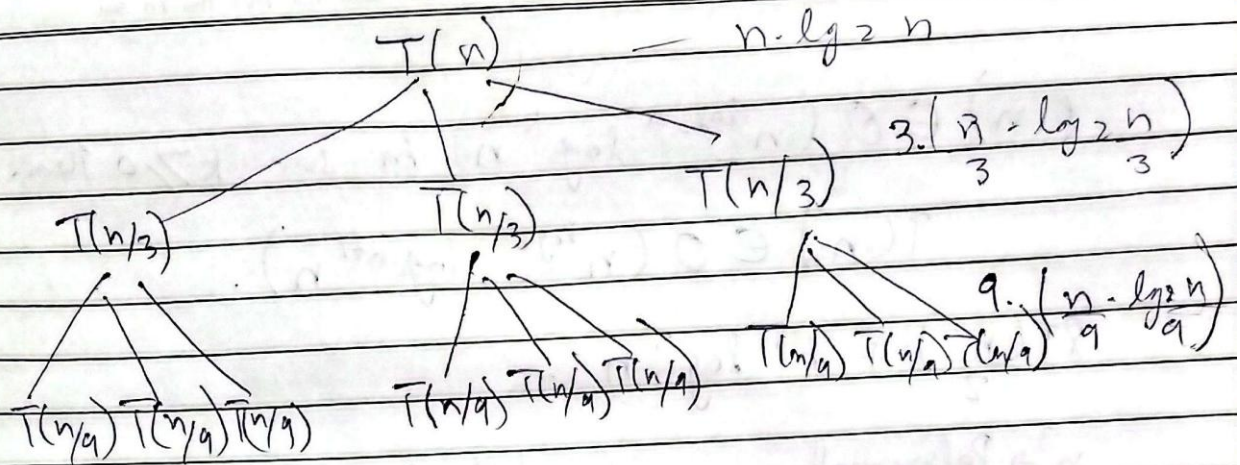
So

$$k \log n - \frac{\log 3}{2} k(k-1) \sim \frac{(\log n)^2}{\log 3} - \frac{(\log n)^2}{2 \log 3} = \frac{(\log n)^2}{2 \log 3}$$

Multiplying by n gives $S(k) = \Theta(n \log n)^2$

$n \cdot \Theta(1) = \Theta(n)$ is less significant therefore

$$T(n) = \Theta(n (\log n)^2)$$



No. of nodes $= 3^k$

Size of each subproblem $= \frac{n}{3^k}$

Cost at each level $= \frac{n}{3^k} \cdot \lg_2 \frac{n}{3^k}$

Total cost

$$3^k \cdot \frac{n}{3^k} \cdot \lg_2 \left(\frac{n}{3^k} \right) = n \cdot (\lg_2 n - \lg_2 (3^k))$$

$$= n (\lg_2 n - k \lg_2 3)$$

$$3^k = n, \quad k = \frac{\lg_2 n}{\lg_2 3}$$

Sum of all levels cost.

$$S = \sum_{i=0}^{k-1} n (\lg_2 n - i \lg_2 3) = n \left(k \lg_2 n - \lg_2 3 \sum_{i=0}^{k-1} i \right)$$

$$= n \left(k \lg_2 n - \lg_2 3 \cdot \frac{k(k-1)}{2} \right)$$

$$k \lg_2 n = \frac{(\lg_2 n)^2}{\lg_2 3}$$

$$\frac{\lg_2^3 n \cdot k(k-1)}{2} = \frac{\lg_2^3 n \cdot k^2}{2} = \frac{\lg_2^3 n}{2 \lg_2 3}$$

Sun Mon Tue Wed Thu Fri Sat

$$S = n \left(\frac{(\log_2 n)^2}{\log_2 3} - \frac{(\log_2 n)^3}{2 \log_2 3} + \dots \right) = \Theta(n (\log_2 n)^2)$$

III. $T(n) = 2T(n/2) + \log n$

$$\log_2 3 \quad \log_2 3$$

(iii) $T(n) = 2T\left(\frac{n}{2}\right) + \log n$

$$T(n) = 2T\left(\frac{n}{2}\right) + \log n \quad \text{eq (i)}$$

$$T\left(\frac{n}{2}\right) = T\left(\frac{n}{2}\right) = 2T\left(\frac{n}{4}\right) + \log \frac{n}{2}$$

substitute in eq (i)

$$T(n) = \left[2T\left(2T\left(\frac{n}{4}\right) + \log \frac{n}{2}\right) \right] + \log n \quad \text{--- (2)}$$

$$2^2 T \cdot \frac{n}{2^2} + 2 \log \frac{n}{2} + \log n$$

$$= 4T\left(\frac{n}{4}\right) + 2(\log n - 1) + \log n$$

$$\Rightarrow 4T\left(\frac{n}{4}\right) + 3 \log n - 2 \quad \text{--- (a)}$$

$$T\left(\frac{n}{4}\right) = 2T\left(\frac{n}{8}\right) + \log \frac{n}{4}$$

substitute in eq (a)

$$4\left(2T\left(\frac{n}{8}\right) + \log \frac{n}{4}\right) + 3 \log n - 2$$

$$8T \cdot \frac{n}{8} + 4 \log \frac{n}{4} + 3 \log n - 2$$

$$T(n) \Rightarrow 8T\left(\frac{n}{8}\right) + 4(\log n - 2) + 3 \log n - 2$$

$$T(n) = 8T\left(\frac{n}{8}\right) + 4 \log n - 8 + 3 \log n - 2$$

$$\Rightarrow 8T\left(\frac{n}{8}\right) + 7 \log n - 10$$

After k expansion

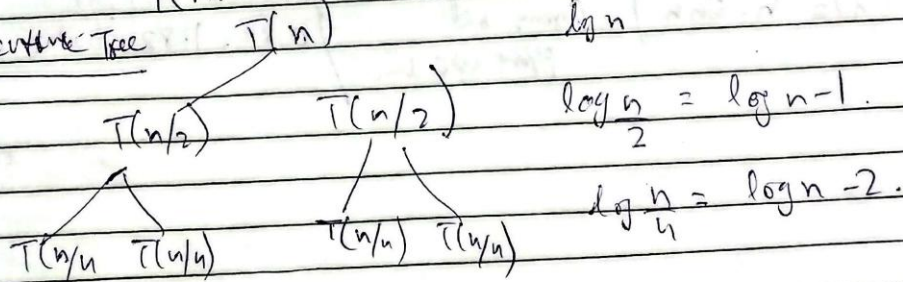
$$T(n) = 2^k T\left(\frac{n}{2^k}\right) + \sum_{i=0}^{k-1} 2^i \log\left(\frac{n}{2^i}\right)$$

$$\sum_{i=0}^{k-1} 2^i (\log n - i) \Rightarrow (\log n) \sum_{i=0}^{k-1} 2^i - \sum_{i=0}^{k-1} i 2^i$$

$$\frac{n}{2^k} = 1, \quad n = 2^k, \quad k = \log_2 n$$

$$T(n) = n T(1) + \Theta(n) = \Theta(n)$$

Recurrence Tree



$$= \log n + 2(\log n - 1) + 4(\log n - 2)$$

$\Theta(n)$

7) Use Master Theorem, to calculate the time complexity of the following

(7.5 Marks)

$$T(n) = 4T(n/2) + n^2$$

I. $T(n) = 2T(n/4) + \sqrt{n}$

II. $T(n) = 4T(n/2) + n^2 \log n$

Master Theorem

i) $4T(n/2) + n^2$

Soln

$$a = 4$$

$$b = 2$$

$$d = 2$$

$$b^d = 2^2 = 4$$

$$a = b^d \Rightarrow 4 = 4$$

2nd case of Master Theorem: $\Theta(n^2)$

$$\Theta(n^d \log n)$$

$$\Rightarrow \Theta(n^2 \log n)$$

ii) $2T\left(\frac{n}{4}\right) + \sqrt{n}$

Soln

$$a = 2, b = 4, n = 1/2$$

$$b^d = 4^{1/2} = 2$$

$$a = b^d \Rightarrow 2 = 2$$

2nd case of Master Theorem

$$\Theta(n^d \log n) \Rightarrow \Theta(n^{1/2} \log n)$$

$$\Theta(\sqrt{n} \log n)$$

$$n \times n \cdot n$$

iii) $T(n) = 4T(n/2) + n^2 \log n$

Soln

$$a=4, b=2, n^{\log_2 4} = n^2$$

$$f(n) = n^2 \log n$$

$$\Theta(n^{\log_2 4} \cdot \log^k n) \text{ at } k=1$$

equal polynomial then $\log^k n$ case

hence

$$\Theta(n^{\log_2 4} \cdot \log^{k+1} n) = \Theta(n^2 \log^2 n)$$

8) Apply the Substitution (Guess & Test) Method on the given recurrence relation to verify whether the provided guess is correct or not. (5 Marks)

I. $T(n) = 4T(n/2) + n^2$

Guess 1: $T(n) = O(n)$

Guess 2: $T(n) = O(n^2)$

Guess and Test Method

$$T(n) = 4T\left(\frac{n}{2}\right) + n^2$$

Guess

$$T(n) = O(n)$$

$$T(n) = 4T\left(\frac{n}{2}\right) + n^2$$

$$T(n) \leq cn$$

Substitute into the recurrence

$$\text{for } n \geq n_0$$
$$T(n) = 4T\left(\frac{n}{2}\right) + n^2$$

$$\leq 4(c \cdot (n/2)) + n^2$$

$$= 2cn + n^2$$

if the guess $T(n) \leq cn$ is to hold
 $2cn + n^2 \leq cn$ for all large n

$$n^2 + (2c - c)n = n^2 + cn \leq 0$$

For any value of c , $n^2 + cn > 0$ for all large n .
Thus the inequality cannot be held for large n .

$T(n) = O(n)$ is false.

Convers $T(n) = O(n^2)$

$$T(n) \leq cn^2$$

For $c > 0$

ad $n \geq n_0$

$$= 4T(n/2) + n^2$$

$$\leq 4 \cdot c \left(\frac{n}{2}\right)^2 + n^2 = 4 \cdot c \frac{n^2}{4} + n^2$$

$$= cn^2 + n^2 = n^2(c+1)$$

$$T(n) \leq cn^2$$

$$(c+1)n^2 \leq cn^2 \text{ for all large } n$$

$c+1 \leq c$, which is impossible

$T(n) = O(n^2)$ is also false.

